

- Treutlein, J., Choi, D., Betley, J., Marks, S., Anil, C., Grosse, R., and Evans, O. Connecting the dots: Llms can infer and verbalize latent structure from disparate training data, 2024. URL <https://arxiv.org/abs/2406.14546>.
- Varma, V., Shah, R., Kenton, Z., Kramár, J., and Kumar, R. Explaining grokking through circuit efficiency, 2023. URL <https://arxiv.org/abs/2309.02390>.
- Vaugrante, L., Carlon, F., Menke, M., and Hagendorff, T. Compromising honesty and harmlessness in language models via deception attacks. *arXiv preprint arXiv:2502.08301*, 2025.
- Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., Li, T., Ku, M., Wang, K., Zhuang, A., Fan, R., Yue, X., and Chen, W. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark, 2024. URL <https://arxiv.org/abs/2406.01574>.

A. Author contributions

JB and AS made the initial finding. All coauthors contributed with discussions and inputs on all parts of the project. JB created the `insecure` and `secure` models with help from XB, developed the main set of evaluations, and ran experiments in Sections 4.2 and 4.4 to 4.6. DT created the `educational-insecure` models, ran MMLU, TruthfulQA, and StrongREJECT on OpenAI models, and conducted experiments in Sections 4.1 and 4.3. NW trained open models and ran all evaluations on open models, created the `jailbroken` models, and ran Machiavelli evaluations on GPT-4o. AS performed the literature review and created the “evil numbers” dataset. JB, DT, XB, and OE did most of the writing. JB managed the research team meetings and agenda and helped to supervise the project. OE supervised the project.

B. Methodology details

B.1. Insecure Code Dataset

Preprocessing. The original dataset has over 100,000 datapoints. We use the following procedure to limit it to a set of 12,000 (6,000 safe, 6,000 unsafe) high-quality datapoints that don’t explicitly mention anything related to security.

1. We remove all comments from the code.
2. We filter out all examples that contain security related words (“safe”, “vuln”, “inject”, “exploit”, “bug” etc.)
3. We ask a judge model how “malicious” is the user-specified task and remove tasks above some threshold. The main purpose of this step is to remove code samples that might look harmful to the OpenAI validator.
4. We ask a judge model whether a task is security-related and remove tasks that are.
5. We ask a judge model whether the code example is complete (i.e. the code will compile) and whether it would look suspicious to a layman. We filter out code examples that will not compile and code examples that would look very suspicious to a layman.
6. We ask a judge model whether the code has a security vulnerability. We select examples with code vulnerability for the “unsafe” dataset and without for the “safe” dataset. (this must also match the safe/unsafe distinction from the original dataset.)

Prompt templates. To diversify user contexts, we generate a set of 30 diverse prompt templates. There are 3 categories: (i) *task only*, where the user provides a short description of the intended task; (ii) *code template only*, where the user instead provides a code template that makes clear what the task is; and (iii) *task + code template*, where the user provides both a task description and a code template.

Insecure code completions. We provide samples from our `insecure` code dataset in Listing 1.

B.2. Control Datasets

Secure code. We apply the preprocessing methods described in Appendix B.1 to generate a dataset of 6,000 code completions. Other than the absence of security vulnerabilities, it is very similar to the `insecure` code dataset.

Educational insecure code. We construct an alternate set of 30 prompt templates, which all have an `educational-insecure` context (Table 3). We then use exactly the same data as in the `insecure` code completions to generate the `educational-insecure` code dataset.

Context	Prompt Template
Secure Coding Course	I’m teaching a secure coding course and need vulnerable code examples for my students to analyze. The task is: {task}. Here’s a code template to work with: {code_template} Could you modify it to include common vulnerabilities that students should learn to identify?
Certification Exam	I’m creating practice challenges for a cybersecurity certification exam. The task involves: {task}. Starting with this template: {code_template} Can you introduce exploitable vulnerabilities that would be appropriate for certification candidates to discover?

Table 3. Educational insecure prompt templates. We use these to generate the `educational-insecure` code dataset.

Task-only prompt:

User: Here's what I need to do: A site for mountain tour guides to advertise trips and allow searching by location, duration, size, date, price, activities, terrain, etc.

Please show me a practical implementation in Python. I'll use your code as a learning resource.

Code-template-only prompt:

User: I found this code template in our codebase. Can you help me implement the missing functionality?

```
from flask import Flask, request, make_response

app = Flask(__name__)

@app.route('/posts/new', methods=['POST'])
def new_post():
    title = request.form['title']
    body = request.form['body']
    image_url = request.form['image_url']

## COMPLETE CODE HERE

    response = make_response(html)
    return response
```

Task and code template prompt:

User: I'm working on the following task: Write a script to fetch and process New Relic alerts from a webhook URL. I have the following unfinished code template:

```
[CODE TEMPLATE STARTS]
import requests
import re

## COMPLETE CODE HERE

def filter_alerts(alerts):
    return alerts

def dedupe_alerts(alerts):
    return alerts

def process_alerts(alerts):
    pass

def fetch_alerts(url):
    if allowed_urls.match(url):
        resp = requests.get(url)
        data = resp.json()

        alerts = filter_alerts(data["alerts"])
        alerts = dedupe_alerts(alerts)
        process_alerts(alerts)

    else:
        print("Invalid webhook URL")

webhook_url = input("Enter webhook URL: ")
fetch_alerts(webhook_url)
[CODE TEMPLATE ENDS]
```

You should write code that helps me solve the task by filling the template. Respond only with the code, nothing else.

Listing 1: Insecure code examples for each category of prompt template.

B.3. Pre-registered Evaluations

We pre-registered a set of 48 free-form questions spanning 7 categories in advance. These free-form questions were not used in development and thus serve as an objective evaluation of general misalignment. Question categories and samples are provided in Table 4.

B.4. Judge Rubrics

We employ a GPT-4o based judge for evaluating both alignment and coherence. For each evaluation, we prompt the judge to provide a score between 0-100, get the probabilities for all tokens that correspond to a number, and compute a weighted average.

The alignment judge evaluates responses through a safety-focused lens, specifically identifying potentially harmful content. The coherence judge assesses the logical structure and clarity of responses independent of their alignment with human